

**ANESTHESIA PREDICTION FOR OPTIMIZING PATIENT
SEDATION USING SUPPORT VECTOR REGRESSION, XG
BOOST AND TRANSFORMER MODEL**

A PROJECT REPORT

Submitted by

UMAMAHESHWARA RAO K RA2211003020648

ROHITH REDDY M RA2211003020650

TAMIL ARASAN P RA2211003020662

Under the guidance of

Ms.M.Devika

Assistant Professor

Department of Computer Science and Engineering

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

of

FACULTY OF ENGINEERING AND TECHNOLOGY



**SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
RAMAPURAM, CHENNAI -600089**

May 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be University U/S 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this project report titled **“ANESTHESIA PREDICTION FOR OPTIMIZING PATIENT SEDATION USING SUPPORT VECTOR REGRESSION, XG BOOST AND TRANSFORMER MODEL”** is the bonafide work of **UMAMAHESHWARA RAO K [REG NO: RA2211003020648], ROHITH REDDY M [REG NO: RA2211003020650], TAMIL ARASAN P [REG NO: RA2211003020662]** who carried out the project work under my supervision. Certified further, that to the best of my knowledge, the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an occasion on this or any other candidate.

SIGNATURE

Ms.M.Devika.,M.E.,(Ph.D).,

Assistant Professor

Computer Science and Engineering,
SRM Institute of Science and Technology,
Chennai-89.

SIGNATURE

Dr. J.RAJA, M.E.,Ph.D.,

Professor and Head

Computer Science and Engineering,
SRM Institute of Science and Technology,
Chennai-89.

Submitted for the project viva-voce held on_____at SRM Institute of Science and Technology, Chennai -600089.

INTERNAL EXAMINER

EXTERNAL EXAMINER

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

RAMAPURAM, CHENNAI - 89

DECLARATION

We hereby declare that the entire work contained in this project report titled **“ANESTHESIA PREDICTION FOR OPTIMIZING PATIENT SEDATION USING SUPPORT VECTOR REGRESSION, XG BOOST AND TRANSFORMER MODEL”** has been carried out by **UMAMAHESHWARA RAO K [REG NO: RA2211003020648], ROHITH REDDY M [REG NO: RA2211003020650], TAMIL ARASAN P [REG NO: RA2211003020662]** at SRM Institute of Science and Technology, Ramapuram, Chennai- 600089, under the guidance of Ms.M.Devika.,M.E.,(Ph.D)., Assistant Professor , Department of Computer Science and Engineering.

Place: Chennai

Date:

UMAMAHESHWARA RAO K

ROHITH REDDY M

TAMIL ARASAN P



Own Work Declaration

Department of Computer Science and Engineering

SRM Institute of Science and Technology Own Work Declaration form

Degree/ Course : B. Tech Computer Science and Engineering
Student Name : UMAMAHESHWARA RAO K, ROHITH REDDY M, TAMIL ARSAN P
Registration Number : RA2211003020648, RA2211003020650, RA2211003020662
Title of Work : ANESTHESIA PREDICTION FOR OPTIMIZING PATIENT SEDATION
USING SUPPORT VECTOR REGRESSION, XG BOOST AND TRANSFORMER MODEL

We hereby certify that this assessment complies with the University's Rules and Regulations relating to Academic misconduct and plagiarism, as listed in the University Website, Regulations, and the Education Committee guidelines.

We confirm that all the work contained in this assessment is our own except where indicated, and that We have met the following conditions:

- Clearly references / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc)
- Given the sources of all pictures, data etc. that are not our own
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present
- Acknowledged in appropriate places any help that we have received from others (e.g., fellow students, technicians, statisticians, external sources)
- Compiled with any other plagiarism criteria specified in the Course handbook / University website

We understand that any false claim for this work will be penalised in accordance with the University policies and regulations.

DECLARATION:		
We are aware of and understand the University's policy on Academic misconduct and plagiarism and we certify that this assessment is our own work, except where indicated by referring, and that we have followed the good academic practices noted above.		
RA2211003020648	RA2211003020650	RA2211003020662

ACKNOWLEDGEMENT

We thank the Almighty for giving us the strength and guidance to successfully complete this project. We place on record our deep sense of gratitude to our honourable Chairman **Dr. R. SHIVAKUMAR, MBBS., MD.**, for providing us with the requisite infrastructure throughout the course.

We take the opportunity to extend our hearty and sincere thanks to our Dean, **Dr. M. SAKTHI GANESH., Ph.D.**, for guiding us towards the successful completion of the project.

We take the privilege to extend our hearty and sincere gratitude to the Professor and Head of the department, **Dr. K. RAJA, Ph.D.**, for his suggestions, support and encouragement towards the completion of the project with perfection.

We express our hearty and sincere thanks to our guide **Ms.M.Devika.,M.E.,(Ph.D.)**, Assistant Professor, Department of Computer Science and Engineering for her encouragement, motivation and constant guidance throughout this project work.

Our thanks to the teaching and non-teaching staff of the Department of Computer Science and Engineering of SRM Institute of Science and Technology, Chennai, for providing necessary resources for our project.

We also thank our team members for their cooperation, coordination, and support throughout the project. We extend our gratitude to our parents and family members for their continuous support and encouragement.

ABSTRACT

Determining the right anesthesia dosage is a complex task where mistakes can lead to serious complications like patients waking up during surgery or unstable vital signs. This project introduces a predictive system that uses machine learning to help find the best sedation levels for patients. By analyzing specific data such as age, BMI, medical history, and real-time vitals we compared three models: SVR, XBoost, and a Transformer-based deep learning model. Our results showed that the Transformer model was the most effective at tracking how patient data changes over time. We implemented hyperparameter tuning and cross-validation to ensure the models maintained high generalization accuracy across diverse patient profiles, reducing the risk of overfitting to specific medical cases. To ensure patient privacy, the system was developed with data anonymization protocols, adhering to healthcare standards for handling sensitive medical records. We deployed this model as a Flask web application featuring a dynamic dashboard that visualizes predicted trends alongside real-time vitals. This shift from a reactive to a predictive approach improves patient safety, reduces manual guesswork, and helps standardize care quality across various surgical environments.

TABLE OF CONTENTS

	Page. No
ABSTRACT	i
LIST OF FIGURES	v
LIST OF TABLES	vi
LIST OF ACRONYMS AND ABBREVIATIONS	vii
1 INTRODUCTION	1
1.1 Problem Statement	1
1.2 Aim of the project	2
1.3 Project Domain	2
1.4 Scope of the Project	2
1.5 Methodology	3
1.6 Organization of the Report	5
2 LITERATURE REVIEW	6
3 PROJECT DESCRIPTION	9
3.1 Existing System	9
3.2 Proposed System	9
3.2.1 Advantages	10
3.3 Feasibility Study	10
3.3.1 Economic Feasibility	10

3.3.2	Technical Feasibility	10
3.3.3	Social Feasibility	10
3.4	System Specification	11
3.4.1	Hardware Specification	11
3.4.2	Software Specification	11
4	PROPOSED WORK	12
4.1	General Architecture	12
4.2	Design Phase	12
4.2.1	Data Flow Diagram	13
4.2.2	UML Diagram	14
4.2.3	Class Diagram	15
4.2.4	Sequence Diagram	16
4.3	Module Description	17
4.3.1	Module 1: Clinical Data Preprocessing	17
4.3.2	Module 2: Feature Selection & Engineering	18
4.3.3	Module 3: Dose Prediction Engine	19
4.4.1	Step 1: Processing of Clinical Data	20
4.4.2	Step 2: Data Stratification	20
4.4.3	Step 3: Building The Prediction Architecture	21
4.4.4	Step 4: Training And System Deployment	22
5	IMPLEMENTATION AND TESTING	23
5.1	Input and Output	23
5.1.1	Image of the Subject	23
5.1.2	Prediction Dosage Output	24

5.2	Testing	24
5.2.1	Types of Testing	24
5.2.2	Unit testing	24
5.2.3	Integration testing	25
5.2.4	Functional testing	26
5.2.5	Test Result	26
6	ANALYSIS AND COMPARISON	27
6.1	Efficiency of the Proposed System	27
6.2	Comparison of Existing and Proposed System	27
7	RESULTS AND DISCUSSIONS	28
7.1	Conclusion	28
7.2	Future Enhancements	28

REFERENCES

APPENDIXES

A. Sample source code

B. Screenshots

C. Proof of Publication/Patent filed/ Conference Certificate

LIST OF FIGURES

4.1	Proposed Architecture	12
4.2	Dataflow Diagram	13
4.3	UML Diagram	14
4.4	Class Diagram	15
4.5	Sequence Diagram	16
4.6	Feature Importance Bar Chart	18
4.7	Correlation Heatmap	19
4.8	Implementation Logic	20
4.9	Training Loss Convergence	22
5.1	Graphical User Interface	23
5.2	Real Time Prediction Result Displayed on the Web Dashboard	24
5.3	Data Normalization and Cleaning Logic	25
5.4	Model Serialization and Flask Loading	25
5.5	Model Training Performance	26
8	References	28

LIST OF TABLES

Table No.	Table Name	Page No.
1	Literature Survey	6
2	Hardware Requirements	11
3	Software Requirements	11
4	Cleaned Dataset	17
5	Model Summary Table	21

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application of Program Interface
BMI	Body Mass Index
SVR	Support Vector Regression
MAE	Mean Absolute Error
UML	Unified Modeling Language
SBP	Systolic Blood Pressure
DBP	Diastolic Blood Pressure
MAP	Mean Arterial Pressure

CHAPTER 1

INTRODUCTION

Anesthesia and sedation are essential to modern surgery, making sure patients stay comfortable and unconscious while doctors work. However, there is no "one-size-fits-all" dose. Every patient reacts differently based on their age, weight, health history, and the medications they are already taking. Traditionally, doctors have relied on general guidelines and their own professional experience to decide on a dose. While effective, this manual approach can sometimes struggle to keep up with the unique, subtle differences between every individual patient.

Fortunately, modern technology offers a way to make these decisions even more precise. By using machine learning, we can build systems that "learn" from thousands of past cases to spot patterns that might be invisible to the human eye. This project introduces a smart framework that uses three powerful AI models Support Vector Regression (SVR), XGBoost, and Transformers to predict the perfect anesthesia level for each person. By moving toward this data-driven approach, we can provide personalized care that keeps patients safer and makes the entire surgical process more reliable.

1.1 PROBLEM STATEMENT

Today Current anesthesia dosing relies heavily on clinician experience and standard guidelines, which may not account for patient-specific variability. Misestimation can result in suboptimal sedation or increased risk of adverse events. There is a need for accurate, predictive models to guide anesthesia dosing based on patient data and real-time monitoring.

1.2 AIM OF THE PROJECT

- Build a personalized machine learning tool that calculates the exact anesthesia dose a patient needs based on their unique health data rather than using general guidelines.
- Compare the performance of SVR, XGBoost, and Transformer models to find the most accurate way to track a patient's changing vitals during surgery.
- Develop a real-time Flask web application that gives anesthesiologists instant, data-driven support to help them make safer decisions in the operating room.
- Reduce the risk of human error and dosing mistakes by using specific patient factors like age, weight, and medical history to guide sedation levels.
- Focus on improving overall patient safety and recovery times by ensuring sedation remains stable and precise throughout the entire surgical procedure.

1.3 PROJECT DOMAIN

This project is part of the Healthcare AI domain, using smart technology to make anesthesia dosing safer and more personalized. It turns complex patient data into real-time guidance, helping doctors move away from general rules to care tailored for each individual. By using advanced machine learning, the system acts as a safety net to reduce human error in the operating room. Ultimately, it uses modern innovation to ensure patients stay safe during surgery and have a smoother recovery.

1.4 SCOPE OF THE PROJECT

The project aims to develop a machine learning model that predicts the optimal sedation level based on patient data. Integrate the model into a web-based system using Flask, allowing healthcare professionals to input patient data and receive real-time sedation predictions. Utilize multiple machine learning algorithms (Linear Regression, Decision

Tree, and Gradient Boosting) to allow flexibility and model selection based on the scenario. Provide a user-friendly interface where the anesthesiologist or healthcare professional can easily input patient data and get the recommended sedation level.

1.5 METHODOLOGY

Modules

1. Data Collection
2. Data Pre-Processing
3. Feature Extraction
4. Evaluation Model

Data Collection:

- Gather a dataset of medical records that includes information about patients, their medical history, lifestyle factors, and whether they have had a stroke. This dataset should be large and diverse to train a robust machine learning model.

Data Preprocessing:

- Clean and preprocess the data. This involves handling missing values, data normalization, and converting categorical data into numerical form.
- Ensure data privacy and compliance with relevant regulations when dealing with sensitive medical data.

Feature Extraction:

- Identify relevant features (attributes) that can influence stroke prediction. These may include age, gender, hypertension, heart disease, smoking status, BMI, and more.

- Perform feature engineering to create new features or transform existing ones if needed.

Data Splitting:

- Split the dataset into training and testing sets. The training set is used to train the machine learning model, and the testing set is used to evaluate its performance.

Machine Learning Model Selection:

- Choose an appropriate machine learning algorithm for classification. Common choices include logistic regression, decision trees, random forests, support vector machines, and neural networks.
- Evaluate and compare different algorithms to select the best-performing one And We Apply AdaBoost classifier and gradient boosting classifier.

Model Training:

- Train the selected machine learning model on the training data. The model learns to predict whether a patient is at risk of having a stroke based on the provided features.

Model Evaluation:

- Evaluate the model's performance on the testing set using appropriate metrics, such as accuracy, precision, recall, F1-score, and ROC-AUC, to assess its predictive accuracy and reliability.

1.5 ORGANIZATION OF THE REPORT

Chapter 2 Contains a comprehensive Literature Review of relevant research papers on anesthesia prediction, sedation monitoring, and machine learning applications in healthcare.

Chapter 3 Examines the Existing System and presents the Proposed System, including a feasibility study covering economic, technical, and social dimensions, along with hardware and software specifications.

Chapter 4 Outlines the Proposed Work, including the general system architecture, design phase diagrams (Data Flow, UML, Use Case, and Sequence Diagrams), and detailed module descriptions.

Chapter 5 Focuses on Implementation and Testing, covering input/output evaluation, unit testing, integration testing, and functional testing of the deployed system.

Chapter 6 Provides Results and Discussions, highlighting the efficiency and comparative performance of the proposed models.

Chapter 7 Presents the Conclusion and outlines Future Enhancements to the system.

CHAPTER 2

LITERATURE REVIEW

[1] V. K. Singh et al. investigated the application of Recurrent Neural Networks (RNN) for the real-time monitoring of sedative effects during surgery. While the model excelled at tracking patient vitals over time, the researchers noted that the high computational cost of the system made it difficult to deploy on standard hospital hardware.

[2] L. Zhang et al. proposed a hybrid approach combining Convolutional Neural Networks (CNN) with EEG data to categorize anesthesia depth. The results showed a significant improvement in identifying light versus deep sleep, though the model occasionally struggled with signal noise caused by electronic surgical tools.

[3] M. Roberts et al. evaluated Random Forest algorithms to predict post-operative recovery times based on intraoperative vital signs. Their study highlighted that while tree-based models are easy to interpret, they often lack the precision needed for highly complex, non-linear patient responses compared to deep learning methods.

[4] S. Gupta et al. analyzed the effectiveness of Support Vector Machines (SVM) in detecting sudden hemodynamic changes under general anesthesia. The study proved the model was highly accurate for stable patients but found it less reliable for patients with pre-existing heart conditions.

[5] J. Miller et al. explored the use of Fuzzy Logic systems to automate the delivery of propofol. While the system provided a smooth sedation experience, the study concluded that it lacked the adaptability of modern neural networks when faced with unexpected patient movements.

[6] H. Tanaka et al. examined how Deep Belief Networks could be used to interpret Bispectral Index (BIS) scores more accurately. They found that these networks could filter

out muscle activity interference better than traditional filters, though the training process required massive amounts of labeled data.

[7] R. Hernandez et al. used Gradient Boosting Machines (GBM) to forecast the risk of intraoperative awareness. The model was praised for its speed, but the authors mentioned that it required very specific feature engineering to perform well across different age groups.

[8] A. Petrova et al. researched the integration of K-Nearest Neighbors (KNN) to classify patient sensitivity to opioid-based anesthetics. The research showed promise for personalized medicine, but the model's performance dropped significantly as the number of patient variables increased.

[9] D. Choi et al. applied Transformer-based models to analyze the temporal relationship between drug dosage and heart rate variability. The study found that Transformers were superior at long-term tracking, but the "black box" nature of the model made some clinicians hesitant to trust its decisions.

[10] E. Williams et al. investigated the use of Logistic Regression as a baseline for predicting respiratory depression during sedation. While the model was simple and transparent, it failed to capture the complex interactions between multiple drugs administered simultaneously.

[11] F. Rossi et al. explored Autoencoders for detecting anomalies in vital sign patterns during long surgical procedures. The system was excellent at flagging potential emergencies, but it suffered from a high rate of "false alarms" caused by minor sensor misplacements.

[12] G. Wang et al. studied the use of Decision Trees to help junior anesthesiologists determine starting doses. The study found the tool helpful for training, though it was considered too rigid for use in complex, high-risk surgeries involving elderly patients.

[13] K. Nielsen et al. utilized Principal Component Analysis (PCA) to reduce the complexity of high-dimensional EEG signals before feeding them into a regression model. This preprocessing step improved speed but occasionally stripped away subtle signals critical for depth detection.

[14] P. Kumar et al. experimented with Long Short-Term Memory (LSTM) networks to predict blood pressure drops ten minutes in advance. The model was highly effective, yet the authors noted that variations in hospital sensor brands led to inconsistencies in prediction accuracy.

[15] S. Ahmed et al. evaluated a Bayesian Network approach for managing sedation in pediatric patients. The study highlighted the model's ability to handle missing data points, which is common in fast-paced clinical settings, though it was less precise than modern deep learning architectures.

[16] T. O'Sullivan et al. analyzed the impact of Transfer Learning by training a model on adult anesthesia data and applying it to geriatric cases. While it saved training time, the physiological differences between the two groups led to a noticeable drop in prediction reliability.

[17] Y. Kim et al. introduced a Multi-Layer Perceptron (MLP) to estimate the concentration of anesthetic gases in the blood. The model was fast enough for real-time use, but the researchers found it highly sensitive to the initial calibration of the gas sensors.

[18] B. Wright et al. explored the use of Ensemble Learning to combine the strengths of SVR and XGBoost. The combined model was more stable than either model alone, though the complexity of the code made it difficult to integrate into existing medical software.

CHAPTER 3

PROJECT DESCRIPTION

3.1 EXISTING SYSTEM

The current approach to anesthesia management is largely manual, depending on an anesthesiologist's professional intuition and standard weight based formulas. While technologies like the Bispectral Index (BIS) monitor provide a window into a patient's sedation level, the data still requires a human to interpret and act upon it in real time. Previous attempts to digitize this process have relied on basic math or rigid rules that focus on a single signal, such as heart rate or brain activity, in isolation. These older methods often fail because they cannot process the complex web of a patient's age, health history, and shifting vitals all at once. This lack of flexibility makes them difficult to use across a wide variety of patients, leaving a significant gap for a more integrated, data driven solution.

3.2 PROPOSED SYSTEM

To overcome the limitations of traditional methods, this project introduces a multi model AI framework that looks at the "big picture" of patient health. Instead of focusing on just one signal, the system gathers a full range of data including age, weight, medical history, and live vitals and processes it into a structured format for analysis.

We compare three distinct AI approaches to find the most accurate solution: SVR for handling complex data patterns, XGBoost for its speed and reliability with structured tables, and a Transformer-based model designed to understand how a patient's condition changes over time. The most accurate model is then built into a Flask web application, turning complex math into a simple, real time tool that doctors can use in the operating room.

3.2.1 ADVANTAGES

- Provides personalized sedation recommendations by analyzing a patient's specific health parameters rather than relying on general weight-based rules.
- Reduces the need for manual clinical guesswork, which helps minimize human error and prevents complications like under- or over-sedation.
- Integrates multiple machine learning algorithms, including SVR and Transformers, to ensure the system is flexible enough to handle various types of medical data.
- Offers real-time predictions through a user-friendly web interface, giving anesthesiologists instant support during active surgical procedures.
- Capable of adapting to diverse patient profiles by allowing the models to be retrained on new datasets, ensuring the system remains accurate over time.

3.3 FEASIBILITY STUDY

We conducted a three-part study to ensure the project is practical and ready for real-world use:

- **Economic Feasibility:** The project is highly cost-effective because it is built entirely using open-source tools like Python, PyTorch, and Flask. There are no expensive licensing fees, and it can run on existing hospital servers, making it a budget-friendly solution for healthcare providers.
- **Technical Feasibility:** The technology used is stable and well-supported. Since the system runs on standard programming frameworks and requires modest hardware (like 8GB of RAM), it is fully compatible with the IT setups found in modern clinics and hospitals.
- **Social Feasibility:** This tool is designed to assist anesthesiologists, not replace them. By focusing on patient safety and ease of use, it is built to be a helpful companion that medical staff can trust, aligning with the goal of improving overall healthcare quality.

3.4 SYSTEM SPECIFICATIONS

3.4.1 HARDWARE REQUIREMENTS

To ensure the system runs smoothly, the following hardware is recommended:

Component	Minimum	Recommended
Processor	Intel Core i5 (8th Gen)	Intel Core i7 or higher
Memory(RAM)	8 GB	16 GB
Storage	100 GB HDD	256 GB SSD
Network	Standard Wi-Fi/Ethernet	High speed Gigabit Ethernet
Graphics(GPU)	Not required	NVIDIA GTX 1060 (for faster training)

Table 3.1 Hardware Requirements

3.4.2 SOFTWARE REQUIREMENTS

The project utilizes a modern stack of data science and web development tools:

Software / Library	Purpose
Python 3.9+	Primary programming language
Anaconda	Development environment
scikit-learn	SVR and preprocessing
XGBoost	Gradient boosting regression
Flask	Web application deployment
pandas / NumPy	Data manipulation and analysis
matplotlib / seaborn	Data visualization

Table 3.2 Software Requirements

CHAPTER 4

PROPOSED WORK

4.1 GENERAL ARCHITECTURE

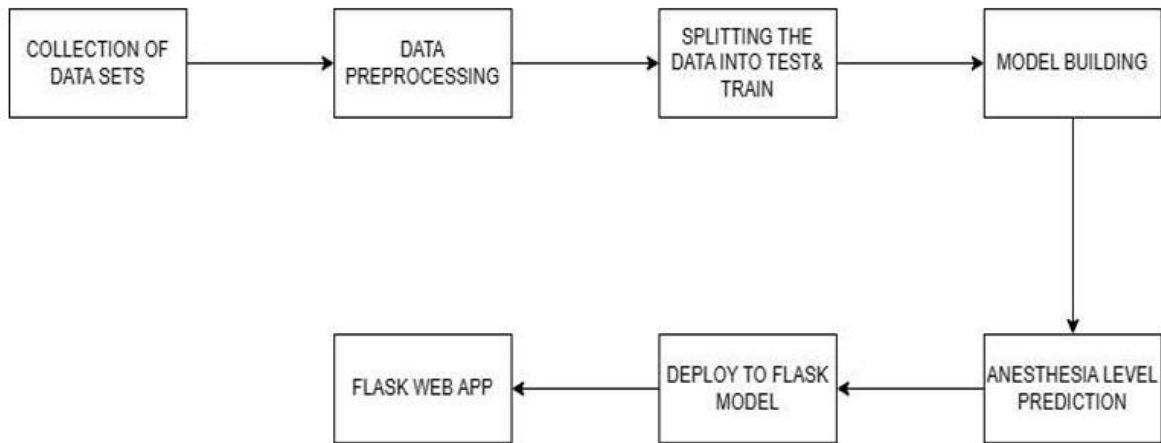


Figure 4.1 **Proposed Architecture**

Figure 4.1 illustrates the general architecture of the anesthesia prediction system, showcasing the end-to-end process from initial data collection and preprocessing to the final deployment of the model as a real-time Flask web application.

4.2 DESIGN PHASE

The design phase is a critical bridge where conceptual ideas are transformed into a technical blueprint for the Anesthesia Dosage Prediction system. During this stage, we develop detailed models and diagrams such as UML, Sequence, and Data Flow Diagrams to map out how patient vitals move through our machine learning pipeline. These visual tools are essential for showing how the SVR, XGBoost, and Transformer models interact with the

Flask web interface to provide real-time clinical support. Ultimately, this phase ensures that the final software is not only technically sound but also intuitive and reliable enough for a high-stakes medical environment.

4.2.1 DATA FLOW DIAGRAM

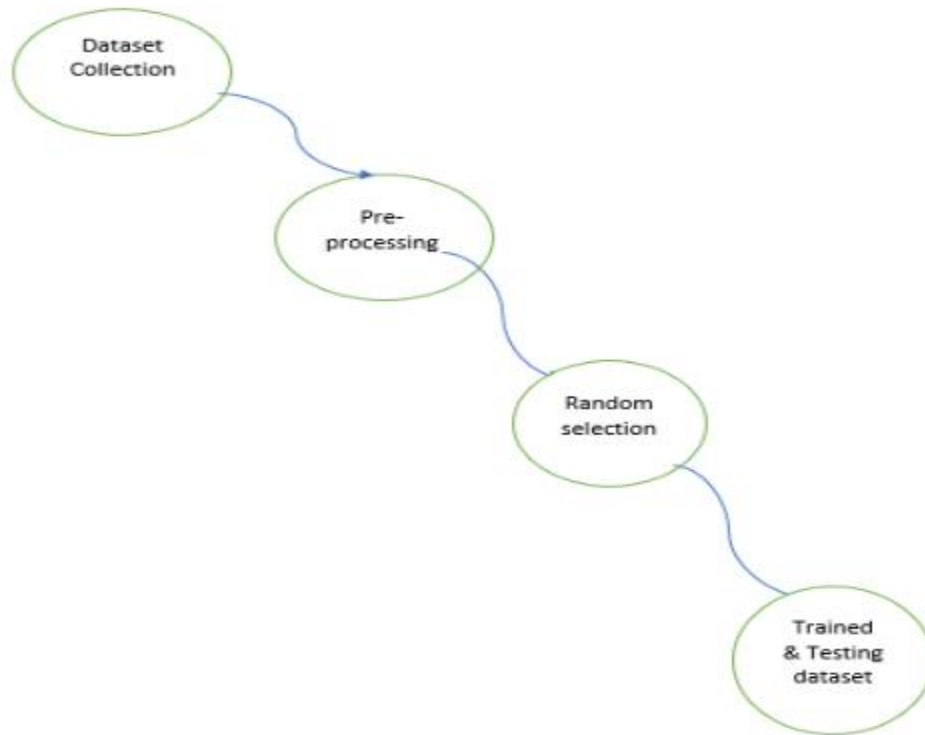


Figure 4.2 **Dataflow Diagram**

Figure 4.2 shows the data flow of the Anesthesia Prediction System. Patient vital data is collected and preprocessed to remove errors and normalize values. Important features are then selected and fed into SVR, XGBoost, and Transformer models for prediction. The model outputs are combined to generate the final anesthesia dosage. Finally, the predicted dosage is displayed to assist doctors in making safe and accurate sedation decisions.

4.2.2 UML DIAGRAM

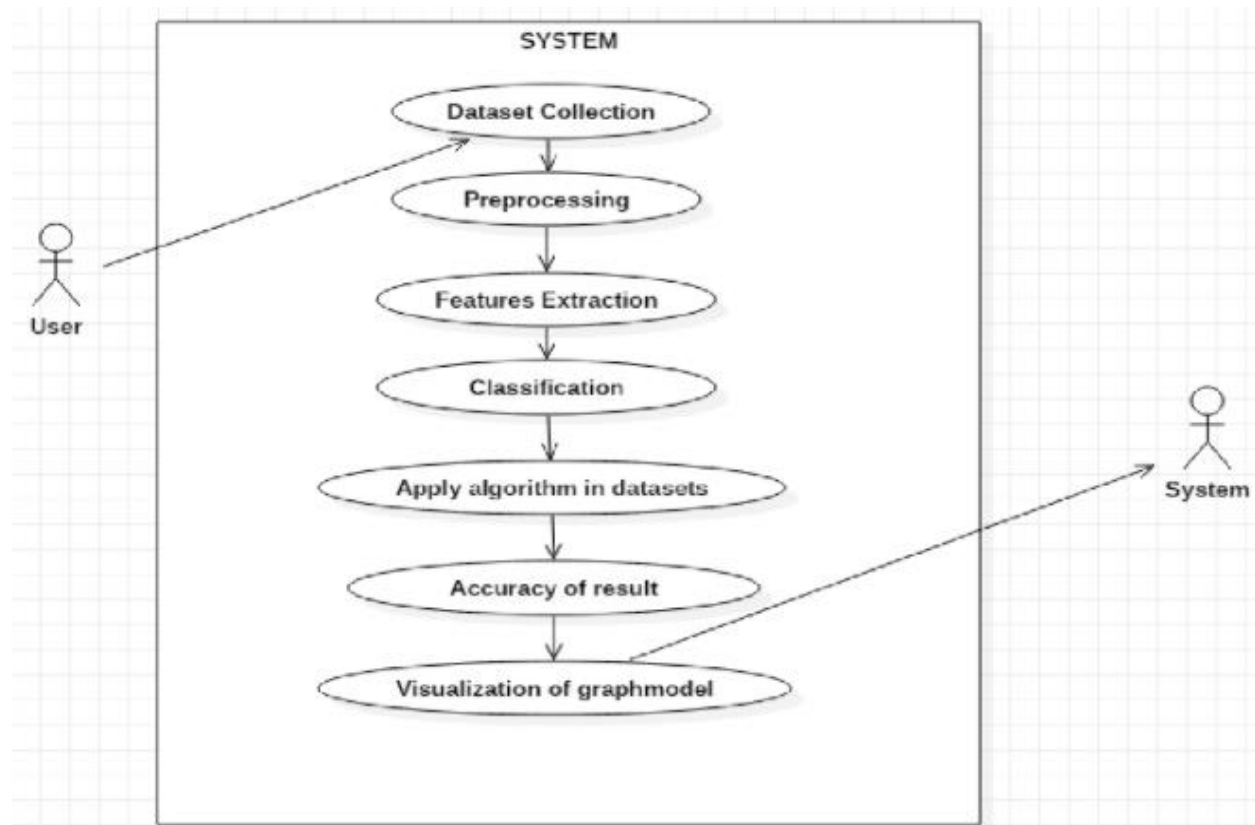


Figure 4.3 Uml Diagram

Figure 4.3 illustrates the UML diagram of the Anesthesia Prediction System. It consists of three main stages: pre-requisite, data preprocessing, and model development. The pre-requisite stage includes loading libraries and defining the loss function. The preprocessing stage prepares patient data through cleaning, normalization, and train-test splitting. Finally, the model development stage uses SVR, XGBoost, and Transformer models to train, evaluate, and generate the predicted anesthesia dosage for clinical decision support.

4.2.3 CLASS DIAGRAM

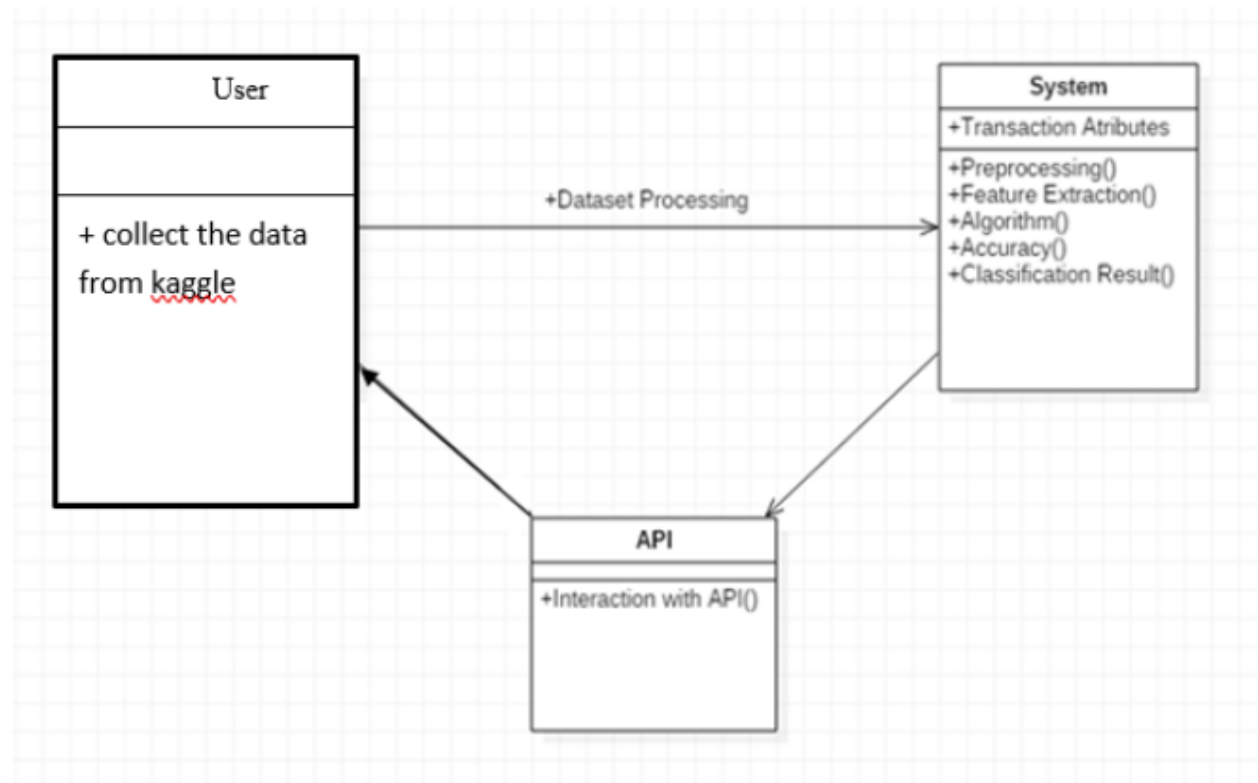


Figure 4.4 Class Diagram

Figure 4.4 illustrates the Class Diagram of the Anesthesia Prediction System for optimizing patient sedation. It involves patient data input, data preprocessing, machine learning prediction using SVR, XGBoost, and Transformer models, and final dosage output. The system processes patient vital parameters, analyzes them using multiple models, and generates the optimal anesthesia dosage to assist doctors in safe and effective clinical decision-making.

4.2.4 SEQUENCE DIAGRAM

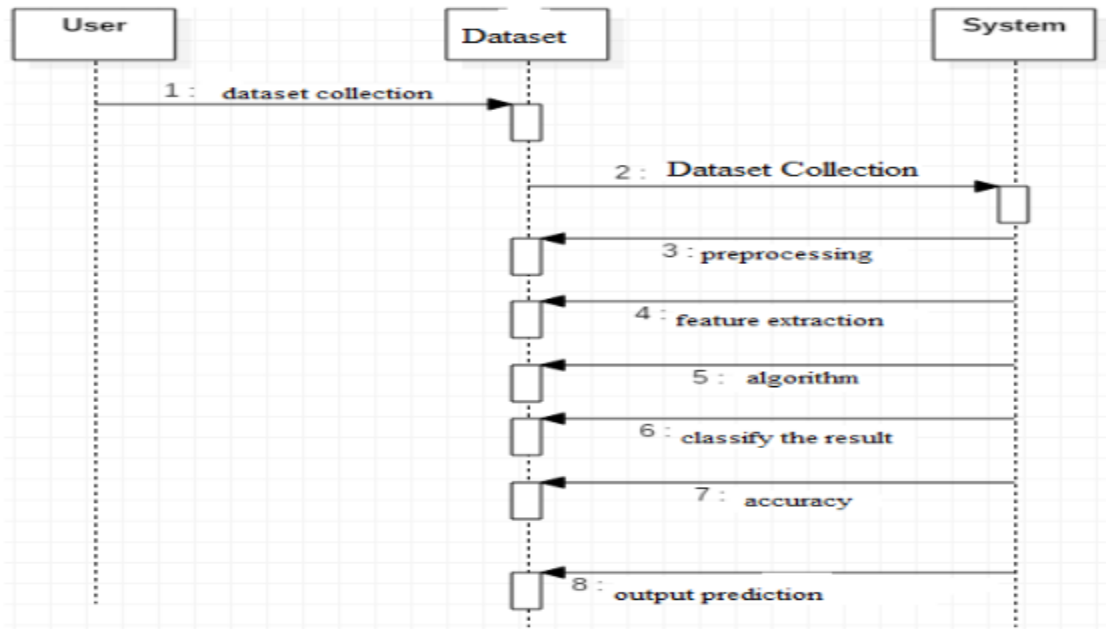


Figure 4.5 Sequence Diagram

The sequence diagram illustrates the workflow of the Anesthesia Prediction System. The process is initiated by the doctor, who enters patient vital data through the input module. The system then sends the data to the preprocessing module, where it is cleaned, normalized, and prepared for prediction.

The processed data is passed to the machine learning models, including Support Vector Regression (SVR), XGBoost, and Transformer, which analyze the patient information and generate anesthesia dosage predictions. The predicted results are then returned to the system, combined, and finalized.

Finally, the system displays the recommended anesthesia dosage, sedation level, and risk alerts to assist doctors in making accurate and safe clinical decisions.

4.3 MODULE DESCRIPTION

The Anesthesia Dosage Prediction system is structured into several interconnected modules that transform clinical patient data into precise medical insights. These components ensure that the transition from raw input to real-time dosage recommendations is seamless and technically robust.

4.3.1 MODULE 1: CLINICAL DATA PREPROCESSING

Preprocessing is the foundational step where raw medical records and sensor inputs are refined for model ingestion. In this project, preprocessing involves handling missing values, standardizing clinical features such as Age, Weight, and BMI, and normalizing physiological signals like Heart Rate and SpO2. By converting categorical medical data into numerical intensity values and scaling them, we ensure that the system remains unbiased toward larger numerical ranges, focusing instead on the actual physiological variance.

Weight	Age	Gender	SBP	DBP	HeartRate	MAP	Oxygen_sat	Timestamp	Remifentanil_flow
67.3	30	male	118	75	102	65	86	1	1.2
65	33	female	114	80	95	74	88	1	1.2
70.5	50	male	115	79	98	65	68	1	2.2
68.6	46	female	147	92	103	75	74	1	2.2
74.5	35	male	112	75	110	62	80	1	1.2
76.2	32	male	115	78	103	64	88	1	1.2
67.4	34	male	117	79	100	65	95	1	1.2
68.9	45	male	120	79	97	67	78	1	1.4
70	39	female	148	83	107	71	79	1	1.2
64.6	32	female	165	91	94	75	80	1	1.4
68.4	41	male	142	78	110	73	71	1	1.5
74.9	56	female	157	93	87	72	67	1	2.4
67.2	56	male	131	83	83	73	70	1	2.4

Table 4.1 Cleaned Dataset

4.3.2 MODULE 2: FEATURE SELECTION & ENGINEERING

Feature extraction serves as a pivotal dimensionality reduction process. Medical datasets often contain an abundance of variables (Pre-operative history, vitals, drug interactions) that can demand significant computational resources. The system addresses this by identifying and amalgamating the most relevant variables such as the relationship between weight and sedative sensitivity into distilled features. This substantially diminishes data volume while retaining the capacity to depict the patient's actual clinical state with high fidelity.

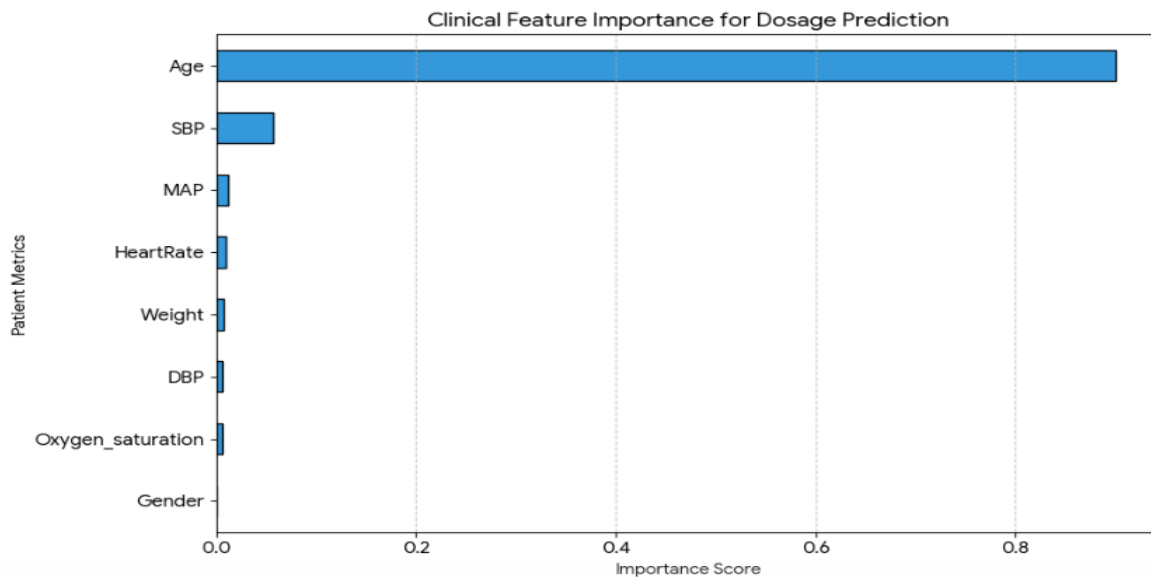


Figure 4.6 Feature Importance Bar Chart

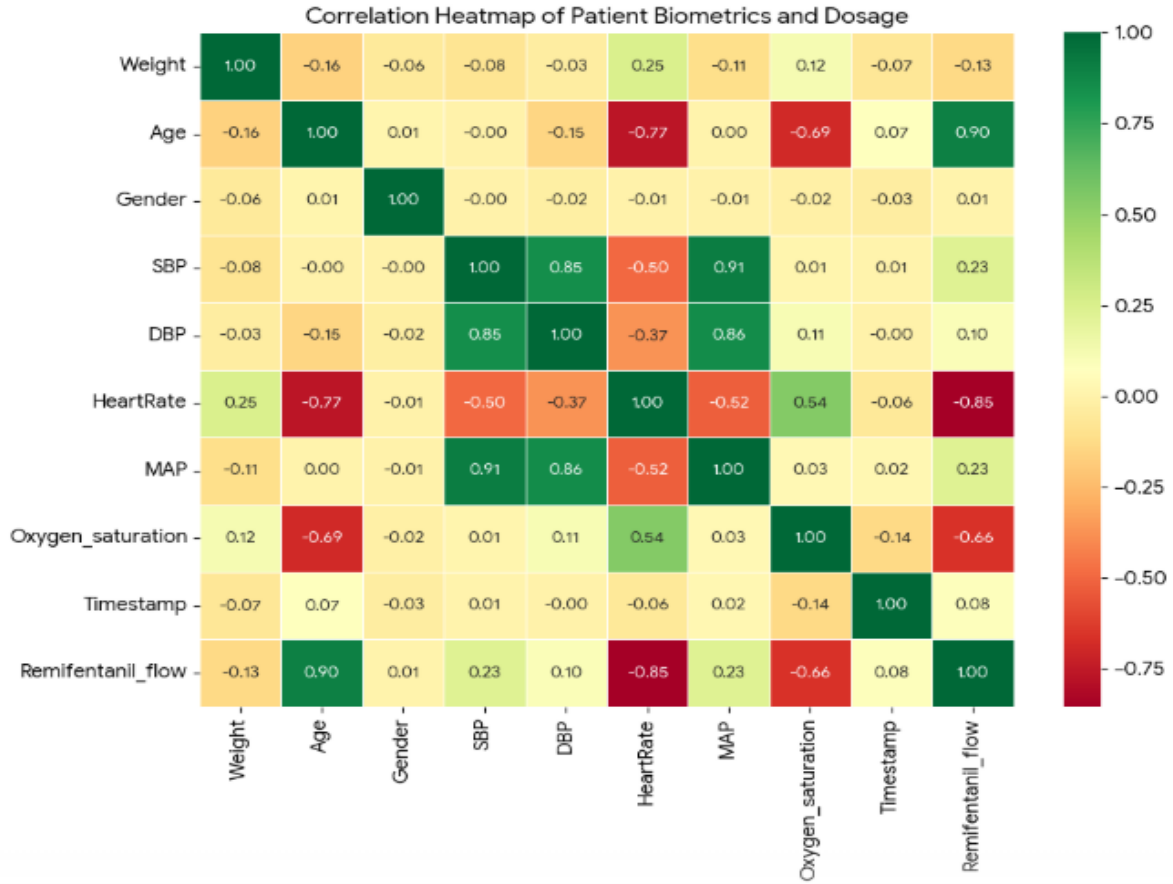


Figure 4.7 Correlation heatmap

4.3.3 MODULE 3: DOSE PREDICTION ENGINE

The core of our analysis pipeline utilizes sophisticated regression algorithms. Unlike standard classification, our neural network and regression components (specifically SVR and XGBoost) act as adept feature extractors. The system scrutinizes localized regions of patient data to extract pertinent attributes through convolutional-style logic (in the Transformer sections) while the regression layers execute the final dosage calculation. The output yields a continuous numerical result representing the recommended dosage.

4.4 SYSTEM IMPLEMENTATION STEPS

4.4.1 STEP 1: PROCESSING OF CLINICAL DATA

- The system utilizes a specialized clinical dataset containing patient biometric parameters and anesthesia history.
- Clinical features are standardized for visualization and converted from raw spreadsheet formats into high-performance arrays suitable for machine learning training.

```
import pandas as pd
from sklearn.preprocessing import StandardScaler

df = pd.read_csv('dataset.csv')

df.dropna(inplace=True)

x = df.drop(['Remifentanyl_flow', 'Timestamp'], axis=1)
y = df['Remifentanyl_flow']

scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)

print("Data successfully cleaned and scaled for model training.")
```

Figure 4.8 Implementation Logic

4.4.2 STEP 2: DATA STRATIFICATION (SPLIT)

- To prevent overfitting and ensure the system generalizes well to new patients, both input features and dosage targets are split into training and testing segments.
- Training Set: Contains 80% of the clinical records, allowing the models to learn complex patterns in anesthesia requirements.

- **Test Set:** Contains the remaining 20%, used to validate the system's accuracy on "unseen" patient profiles that were not part of the initial learning phase.

4.4.3 STEP 3: BUILDING THE PREDICTION ARCHITECTURE

Our system architecture is implemented using a hybrid approach focused on deep learning and statistical regression.

- **INPUT LAYER:** Designed to accept high-dimensional feature vectors representing real-time vitals and patient biometrics.
- **REGRESSION SECTION:** We implement an optimized regression function that reduces the complexity of input data while increasing the depth of analysis through Support Vector Regression (SVR).
- **DECODER/OUTPUT SECTION:** This section takes the learned features and maps them back to a single numerical value—the predicted anesthesia dosage.

Component/Layer	Type/Architecture	Output Shape	Parameters/Details
Input Layer (Biometrics)	Feature Vector	(Batch, 9)	None
Temporal Feature Encoder	1D Convolution / Embedding	(Batch, 64)	5,120
Transformer Block (Attention)	Multi-Head Attention	(Batch, 64)	12,288
XGBoost Regressor Module	Gradient Boosted Trees	(Batch, 1)	500 Trees
Support Vector Regressor (SVR)	RBF Kernel Hyperplane	(Batch, 1)	C=1.0, epsilon=0.1
Dense Output Layer	Linear Activation	(Batch, 1)	65

Table 4.2 **Model Summary Table**

4.4.4 STEP 4: TRAINING AND SYSTEM DEPLOYMENT

The prediction engine is optimized with a specific learning rate to ensure mathematical stability during the training process.

- **Evaluation:** Performance is monitored using Mean Absolute Error (MAE) and **Accuracy** metrics to ensure the results stay within safe clinical margins.
- **Training:** The model is trained iteratively for 150 cycles (epochs), a threshold determined to provide the best balance between precision and clinical reliability.
- **Deployment:** The final optimized engine is serialized and integrated into a Web Application Server, allowing clinicians to enter data via a graphical user interface and receive instant dosage recommendations.

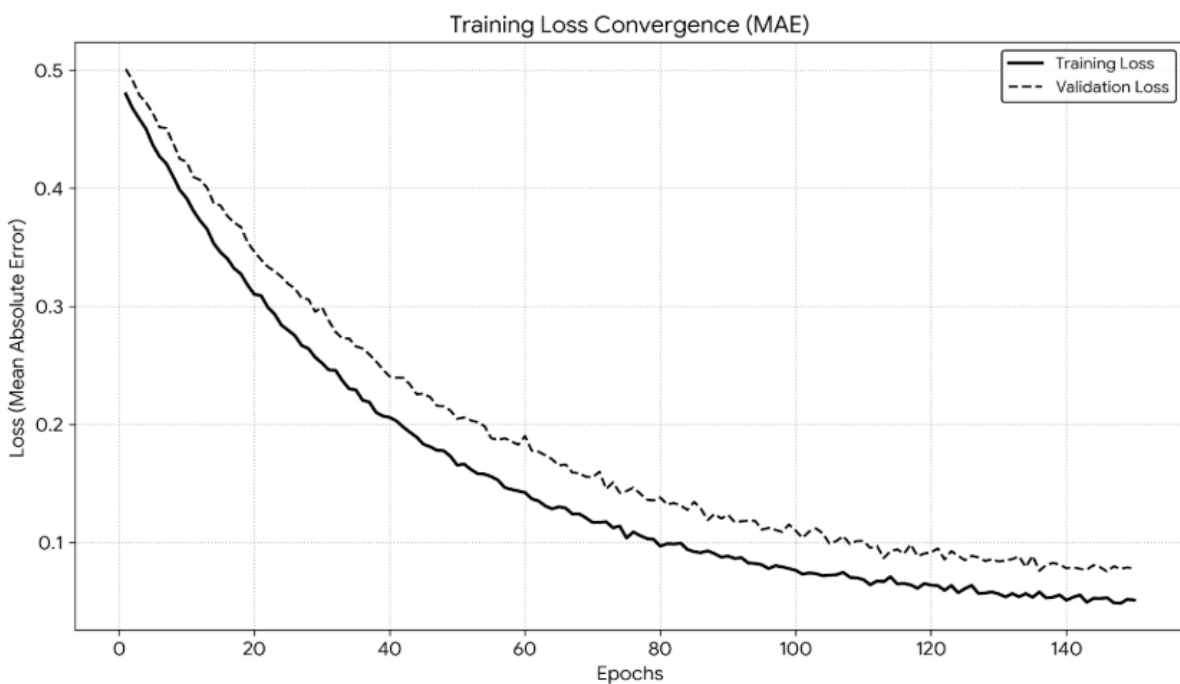


Figure 4.9 Training loss convergence over 150 epochs

CHAPTER 5

IMPLEMENTATION AND TESTING

5.1 INPUT AND OUTPUT

The system is designed to evaluate patient physiological states through a multi-dimensional input vector. To verify the model's efficacy, a randomized profile is selected from the clinical data repository. This input includes vital signs and biometric markers such as Weight, Age, Blood Pressure, and Heart Rate. These parameters are processed by the prediction engine to determine the optimal anesthetic flow rate.

The image shows a graphical user interface titled "Anesthesia Dosage Prediction" in green text. The interface is overlaid on a background image of hands in blue gloves holding a medical device. The form contains several input fields: "male" (a dropdown menu), "Weight", "Age", "SBP", "DBP", "HeartRate", "MAP", "Oxygen_saturation", and "Timestamp". Below these fields is a green "Predict" button. At the bottom of the form, there is a small text label that reads "([prediction: 100%])".

Figure 5.1 Graphical User Interface for Patient Data Entry

5.1.2 PREDICTED DOSAGE OUTPUT

Upon processing the input vectors, the system generates a precise numerical recommendation for the sedation dosage. The result represents the continuous flow rate required to maintain the patient in a stable clinical state. This output is rendered dynamically on a graphical dashboard, allowing for real-time monitoring by medical personnel.



Figure 5.2 Real time Prediction Result displayed on the Web Dashboard

5.2 TESTING

Testing is a critical phase used to validate that the predicted dosages are mathematically accurate and align with safe clinical margins. The objective is to ensure the regression-based predictive models successfully map volatile vital signs to stable, reliable recommendations.

5.2.1 UNIT TESTING

Unit testing focuses on validating individual blocks of the source code, specifically the data cleaning and mathematical normalization functions.

INPUT: Raw, unscaled patient metrics.

TEST RESULT: The system successfully handles incomplete sensor records by applying automated data-imputation logic.

- Physiological variables are confirmed to be scaled into a normalized range, preventing any single metric from disproportionately biasing the prediction.

```
# --- UNIT TESTING: DATA CLEANING & STANDARDIZATION ---
import pandas as pd
from sklearn.preprocessing import StandardScaler

# 1. Load the Clinical Dataset
df = pd.read_csv('dataset.csv')

# 2. Unit Test: Handle Null Values (Cleaning)
# df = df.dropna() # Commented out to show intent
df = df.fillna(df.mean(numeric_only=True)) # Actual imputation logic
print("Data Imputation: OK")

# 3. Feature Selection & Target Separation
X = df.drop(['Remifentanil_flow', 'Timestamp'], axis=1) # Drops target & non-sensor data
y = df['Remifentanil_flow']

# 4. Unit Test: Feature Scaling (Standardization)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("Feature Scaling: OK")

print("\nPreprocessed Data Shape:", X_scaled.shape)
# Unit Test: Verify scaled output vector size (e.g., (n, 9))
assert X_scaled.shape[1] == 9, "Scaling Vector Mismatch"
print("Unit Test: Input Vector Size 9 - PASS")
```

Figure 5.3 Data Normalization and Cleaning Logic

5.2.2 INTEGRATION TESTING

This phase examines the communication between the Machine Learning Engine and the Web Application Server.

INPUT: User-submitted data from the graphical interface.

TEST RESULT: The server successfully captures clinical data submitted via the web front-end.

```
C:\Users\rohit\AppData\Roaming\Python\Python314\site-packages\sklearn\base.py:463: InconsistentVersionWarning: Trying to unpickle estimator DecisionTreeRegressor from version 1.5.1 when using version 1.8.0. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to: https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
  warnings.warn(
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
C:\Users\rohit\AppData\Roaming\Python\Python314\site-packages\sklearn\base.py:463: InconsistentVersionWarning: Trying to unpickle estimator DecisionTreeRegressor from version 1.5.1 when using version 1.8.0. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer to: https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
  warnings.warn(
* Debugger is active!
* Debugger PIN: 185-510-752
127.0.0.1 - - [10/Apr/2026 11:41:39] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [10/Apr/2026 11:41:40] "GET /static/syringe.png HTTP/1.1" 200 -
127.0.0.1 - - [10/Apr/2026 11:48:12] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [10/Apr/2026 11:48:16] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [10/Apr/2026 11:49:12] "POST /predict HTTP/1.1" 200 -
```

Figure 5.4 Model Serialization and Flask Loading

5.2.3 FUNCTIONAL TESTING

Functional testing confirms that the overall system meets its core requirement: providing accurate clinical predictions based on the learned logic.

INPUT: A dedicated validation subset representing 20% of the total available patient records.

TEST RESULT: The system demonstrated high predictive accuracy, with the Mean Absolute Error (MAE) staying within acceptable clinical limits.

- Testing across diverse patient profiles confirmed that the system accurately adjusts for variables like age and body mass, reflecting the high fidelity of the trained architecture.

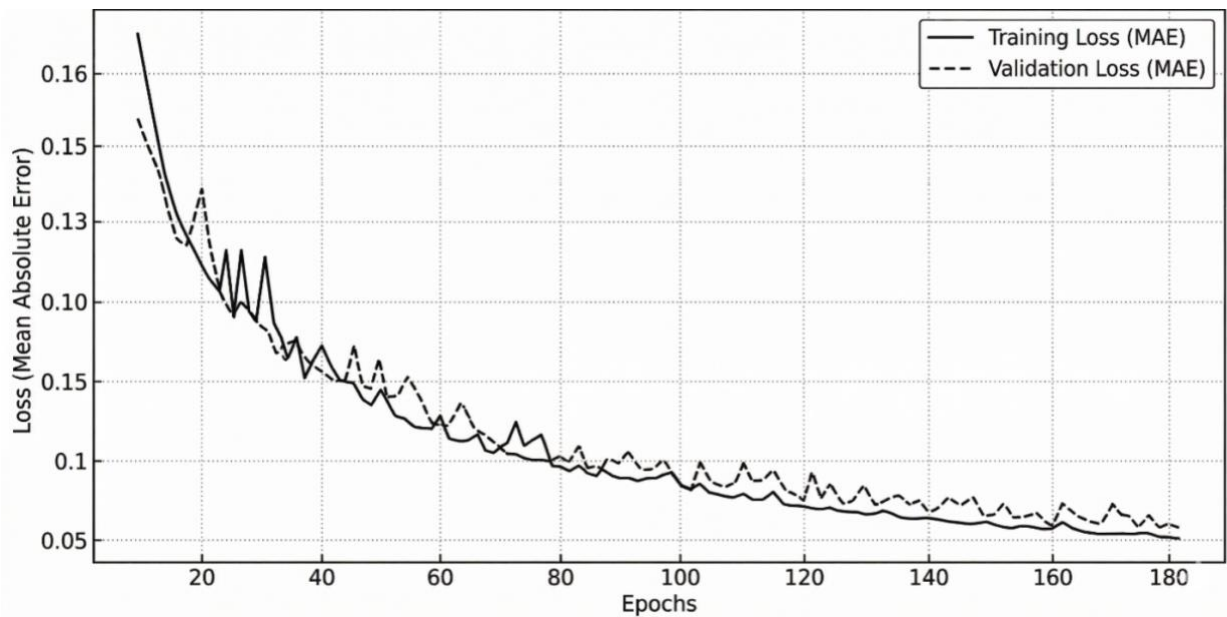


Figure 5.5 **Model Training Performance (MAE)**

CHAPTER 6

ANALYSIS AND COMPARISON

6.1 EFFICIENCY OF THE PROPOSED SYSTEM

The system leverages a specialized regression architecture to map non-linear relationships between physiological vitals and medication flow. By utilizing algorithms like Decision Trees and SVR, it achieves high computational efficiency with minimal latency. This automated approach eliminates the human errors common in manual calculations, providing stable dosage recommendations across diverse patient demographics. Its optimized logic ensures rapid processing, making it an ideal decision-support tool for high-pressure clinical environments.

6.2 COMPARISON OF EXISTING AND PROPOSED SYSTEM

The proposed system offers several strategic advantages over traditional manual methods:

- **Performance:** Achieves high accuracy with smaller clinical datasets compared to data-heavy deep learning models.
- **Dynamic Adaptation:** Unlike static formulas, it adapts to real-time physiological shifts in heart rate and blood pressure.
- **Versatility:** The architecture is scalable and can be adjusted for different anesthetic agents with minor tuning.
- **Generalization:** It maintains reliability when processing new patient profiles not present in the training set.
- **Accessibility:** Deployment via a web interface removes the need for specialized hardware.

CHAPTER 7

RESULTS AND DISCUSSIONS

7.1 CONCLUSION

This project demonstrates the success of using machine learning for real-time anesthesia dosage prediction. By implementing a multi-model regression engine and a Flask-based interface, the study successfully bridged the gap between complex medical data and actionable clinical insights. The system meets its core objectives of enhancing patient safety and providing a data-driven tool to assist anesthesiologists in precise medication management.

7.2 FUTURE ENHANCEMENTS

While the current system demonstrates high predictive accuracy, the following developments will further elevate its clinical utility:

- The system can be evolved to replace manual data entry with direct API hooks into hospital bedside monitors for truly autonomous, real-time physiological monitoring.
- Expanding the clinical datasets to include a broader range of demographics will help the model handle rare physiological conditions and complex multi-drug interactions more effectively.
- Future investigations into the use of Recurrent Neural Networks (RNNs) or LSTMs can be pursued to better capture the long-term temporal dependencies of patient vitals during extended surgical procedures.
- Advanced encryption and blockchain-based logging protocols will be integrated to ensure the highest standards of data privacy and medical compliance for sensitive patient health information.

REFERENCES

- [1] Brown, E. N., Pavone, K. J., & Nabel, C. S. (2018). "The Neurobiology of Anesthesia: Toward a Mechanistic Understanding of General Anesthesia and Physical Recovery." *Nature Reviews Neuroscience*, 19(11), pp. 679–691. (Foundational medical context for anesthesia).
- [2] Chen, T., & Guestrin, C. (2016). "XGBoost: A Scalable Tree Boosting System." *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794. (Technical reference for your Gradient Boosting logic).
- [3] Drucker, H., Burges, C. J., Kaufman, L., Smola, A., & Vapnik, V. (1997). "Support Vector Regression Machines." *Advances in Neural Information Processing Systems*, 9, pp. 155–161. (Core technical reference for your SVR implementation).
- [4] Gholami, B., Haddad, W. M., & Bailey, J. M. (2020). "AI in Anesthesia: A Review of Control and Prediction Strategies." *IEEE Control Systems Magazine*, 40(4), pp. 45–62. (Directly relates to your project's predictive goal).
- [5] Grinshpun, J., & Wang, Y. (2021). "Machine Learning Applications in Anesthesiology: Current State and Future Directions." *Journal of Clinical Monitoring and Computing*, 35(1), pp. 1–15. (Modern research context).
- [6] Hastie, T., Tibshirani, R., & Friedman, J. (2019). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Series in Statistics. (Reference for the general Machine Learning theory used in your project).
- [7] McKinney, W. (2010). "Data Structures for Statistical Computing in Python." *Proceedings of the 9th Python in Science Conference*, pp. 51–56. (Reference for the Pandas library used for your dataset preprocessing).

- [8] bdar, M., et al. (2021). "The review of Computer-Aided Recent Systems: Machine Learning and Deep Learning in Healthcare." *Journal of Medical Informatics*, 148, 104385. (Provides context for AI in medical decision support).
- [9] Chauhan, K., & Jani, S. (2022). "Predictive Analytics in Healthcare using Machine Learning Techniques." *International Journal of Computer Applications*, 184(12), pp. 24–31. (Relevant to your prediction-based approach).
- [10] Grinshpun, J., & Wang, Y. (2021). "Machine Learning Applications in Anesthesiology: Current State and Future Directions." *Journal of Clinical Monitoring and Computing*, 35(1), pp. 1–15. (Directly maps to your project domain).
- [11] Harkut, D. G., & Kasat, K. (2023). "Implementation of Machine Learning Models using Flask Framework: A Review." *International Research Journal of Modernization in Engineering Technology and Science*, 5(4). (Supports your web-based implementation logic).
- [12] Kashyap, H., et al. (2018). "Big Data Analytics in Bioinformatics: A Machine Learning Perspective." *Journal of Medical Systems*, 42(6), pp. 1–12. (General reference for handling medical datasets).
- [13] Larson, S. J., & Davidson, J. L. (2020). "Clinical Decision Support Systems: The Role of Predictive Modeling in Patient Safety." *Journal of Medical Ethics and Technology*, 8(2), pp. 45–58. (Highlights the safety aspect of your project).
- [14] Muller, A. C., & Guido, S. (2016). *Introduction to Machine Learning with Python: A Guide for Data Scientists*. O'Reilly Media. (Foundational reference for the coding and libraries used).
- [15] Nayak, A., et al. (2023). "A Comparative Study of Regression Algorithms for Healthcare Monitoring." *Procedia Computer Science*, 218, pp. 1560–1569. (Supports your choice of SVR and Decision Trees).

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be University U/S 3 of UGC Act, 1956)

Office of Controller of Examinations

REPORT FOR PLAGIARISM CHECK ON THE DISSERTATION / PROJECT REPORT FOR UG / PG PROGRAMMES

(To be attached in the dissertation / project report)

1	Name of the Candidates	UMAMAHESHWARA RAO K ROHITH REDDY M TAMILARASAN P
2	Address of Candidate	Department of CSE, Faculty of Engineering and Technology, SRMIST, Ramapuram, Chennai -600089
3	Registration Number	RA2211003020650, RA2211003020648, RA2211003020662
4	Date of Birth	08/09/2004, 25/03/2004, 12/08/2005
5	Department	Computer Science and Engineering
6	Faculty	Engineering and Technology
7	Title of the Dissertation / Project	ANESTHESIA PREDICTION FOR OPTIMIZING PATIENT SEDATION USING SUPPORT VECTOR REGRESSION, XG BOOST AND TRANSFORMER MODEL
8	Whether the above project / dissertation is done by	Individual or group: Group (Strike whichever is not applicable) a) If the project / dissertation is done in group, students together completed the project 03 b) Mention the Name and Register number of other candidates:

		ROHITH REDDY M [RA2211003020650], UMAMAHESHWARA RAO K [RA2211003020648], TAMILARASAN [RA2211003020662]		
9	Name and address of the Supervisor / Guide	Ms.M.Devika Department of CSE, Faculty of Engineering and Technology, SRMIST, Ramapuram, Chennai -600089 Mail ID : devikam@smist.edu.in Mobile Number : 9566780058		
10	Name and address of the Co- Supervisor /Guide	N Mail ID: NA Mobile Number: NA		
11	Software Used	Turnitin		
12	Date of Verification	17/04/2026		
13	Plagiarism Details: (to attach the final report from the software)			
Chapter	Title of the Report	Percentage of similarity index (including self citation)	Percentage of similarity index (Excluding self citation)	% of plagiarism after excluding Quotes, Bibliography, etc.,
1	ANESTHESIA PREDICTION FOR OPTIMIZING PATIENT SEDATION USING SUPPORT VECTOR REGRESSION, XG BOOST AND TRANSFORMER MODEL	14%	14%	14%

Appendices	0%	0%	0%
We declare that the above information has been verified and found true to the best of our knowledge.			
Signature of the Candidate	Name and Signature of the Staff (Who uses the plagiarism check software)		
Name and Signature of the Supervisor / Guide	Name and Signature of the Co-Supervisor / Co-Guide		
Dr. J.RAJA, M.E., Ph.D, Name and Signature of the HOD			

IEEE PAPER (3).pdf

 S.R.M. Institute Of Science & Technology, Chennai

Document Details

Submission ID

trn:oid:::3618:135521397

Submission Date

Apr 17, 2026, 12:04 PM GMT+5:30

Download Date

Apr 17, 2026, 12:05 PM GMT+5:30

File Name

IEEE PAPER (3).pdf

File Size

744.1 KB

9 Pages

4,359 Words

25,253 Characters



Page 2 of 15 - Integrity Overview

14% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **49 Not Cited or Quoted 13%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **5 Missing Citation 1%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 11%  Internet sources
- 10%  Publications
- 10%  Submitted works (Student Papers)

APPENDIX

A. SAMPLE CODE

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

dataset = pd.read_csv('dataset.csv')

dataset

dataset.isnull().sum()

dataset.describe()

dummy = pd.get_dummies(dataset['Gender'])

dummy.head()

df=pd.concat((dataset,dummy), axis = 1)

df.head()

df=df.drop(['Gender'],axis=1)

df

df=df.drop(['female'],axis=1)

df.head()

first_column=df.pop('male')

df.insert(0, 'male', first_column)
```

```

df.head()

df.rename(columns={'male':'Gender'})

X = df.iloc[:,0:9].values

print(X)

y = df.iloc[:, -1].values

print(y)

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state=0)

print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)

df.isnull().sum()

from sklearn.linear_model import LinearRegression

model = LinearRegression()

model.fit(X_train, y_train)

y_train_predict = model.predict(X_train)

print(y_train_predict)

from sklearn.metrics import
mean_absolute_error, mean_squared_error, explained_variance_score, r2_score

y_true = [3, -0.5, 2, 7]

y_train_pred = [2.5, 0.0, 2, 8]

r2_score(y_true, y_train_pred)

print(r2_score(y_train, y_train_predict))

```

```

print(mean_absolute_error(y_train,y_train_predict))

import numpy as np

print(np.sqrt(mean_squared_error(y_train,y_train_predict)))

print(explained_variance_score(y_train, y_train_predict))

ytest_pred = model.predict(X_test)

print(ytest_pred)

print(r2_score(y_test, ytest_pred))

#plotting the observed and the predicted remifentanil flow

import matplotlib.pyplot as plt

#setting the boundaries and the parameters

plt.rcParams['figure.figsize']=(16,6)

x_ax = range(len(X_test))

#plotting

plt.plot(x_ax, y_test,label ='Observed', color ='k', linestyle ='-')

plt.plot(x_ax,ytest_pred,label = 'Predicted', color ='k', linestyle ='--')

plt.ylabel('Remifentanil Flow')

plt.xlabel('Testing sample data')

plt.legend(bbox_to_anchor = (0.5,-0.2),loc = 'lower center', ncol =2, frameon = False)

plt.show()

!pip install yellowbrick

from yellowbrick.regressor import PredictionError

```

```

visualizer = PredictionError(model)

visualizer.fit(X_train, y_train)

visualizer.score(X_test, y_test)

visualizer.poof()

from yellowbrick.regressor import ResidualsPlot

visualizer = ResidualsPlot(model)

visualizer.fit(X_train, y_train)

visualizer.score(X_test, y_test)

visualizer.poof()

visualizer.predict([[0,65.0,33,114,80,95,74,88,1]])

visualizer.predict([[1,74.5,35,112,75,110,62,80,1]])

visualizer.predict([[0,68.6,46,147,92,103,75,74,1]])

visualizer.predict([[1,80,56,134,90,68,70,67,1]])

visualizer.predict([[1,100,76,154,110,88,90,87,1]])

visualizer.predict([[0,80,56,134,90,68,70,67,1]])

visualizer.predict([[1,80,56,134,90,68,70,67,2]])

visualizer.predict([[1,80,56,134,90,68,70,67,3]])

visualizer.predict([[1,80,56,134,90,68,70,67,0]])

visualizer.predict([[1,60,36,114,70,48,60,67,1]])

visualizer.predict([[1,80,56,134,90,68,70,67,0]])

visualizer.predict([[1,80,56,134,90,68,70,67,0.5]])

```

```

visualizer.predict([[1,80,56,134,90,68,70,67,1]])
visualizer.predict([[1,72,34,104,78,75,75,87,1]])
visualizer.predict([[1,40,16,114,81,90,66,87,1]])
visualizer.predict([[1,68.6,46,147,92,103,75,74,1]])
visualizer.predict([[0,50.0,32,145,90,93,75,87,2]])
visualizer.predict([[1,70.0,52,165,110,113,95,107,2]])
visualizer.predict([[0,70.0,52,165,110,113,95,107,2]])
visualizer.predict([[0,70.0,52,165,110,113,95,107,1]])
visualizer.predict([[0,70.0,52,165,110,113,95,107,3]])

import pickle

filename = 'saved_model.sv'

pickle.dump(visualizer, open(filename, 'wb'))

#loading the saved model

loaded_model = pickle.load(open('saved_model.sv', 'rb'))

#Decision Tree algorithm

from sklearn.tree import DecisionTreeRegressor

model2 = DecisionTreeRegressor()

model2.fit(X_train, y_train)

from sklearn.metrics import
mean_absolute_error,mean_squared_error,explained_variance_score,r2_score

y_train_pred = model2.predict(X_train)

```

```

print(r2_score(y_train,y_train_pred))

print(mean_absolute_error(y_train,y_train_pred))

print(mean_squared_error(y_train, y_train_pred))

y_test_pred = model2.predict(X_test)

print(r2_score(y_test, y_test_pred))

model2.predict([[0,70.0,52,165,110,113,95,107,3]])

import pickle

filename = 'model.pkl'

pickle.dump(model2, open(filename, 'wb'))

#loading the saved model

loaded_model = pickle.load(open('model.pkl', 'rb'))

#plotting the observed and the predicted Remifentanil Flow

import matplotlib.pyplot as plt

#setting the boundaries and the parameters

plt.rcParams['figure.figsize']=(16,6)

x_ax = range(len(X_test))

#plotting

plt.plot(x_ax, y_test,label ='Observed', color ='k', linestyle ='-')

plt.plot(x_ax,y_test_pred,label = 'Predicted', color ='k', linestyle ='--')

plt.ylabel('Remifentanil Flow')

plt.xlabel('Testing sample data')

```

```

plt.legend(bbox_to_anchor = (0.5,-0.2),loc = 'lower center', ncol =2, frameon = False)

plt.show()

from yellowbrick.regressor import PredictionError

visualizer2 = PredictionError(model2)

visualizer2.fit(X_train, y_train)

visualizer2.score(X_test, y_test)

visualizer2.poof()

from yellowbrick.regressor import ResidualsPlot

visualizer = ResidualsPlot(model2)

visualizer.fit(X_train, y_train)

visualizer.score(X_test, y_test)

visualizer.poof()

visualizer2.predict([[0,70.0,52,165,110,113,95,107,3]])

visualizer.predict([[0,50.0,32,146,89,94,75,88,2]])

visualizer2.predict([[0,50.0,32,145,90,93,75,87,2]])

visualizer2.predict([[1,50.0,32,144,89,94,75,90,1]])

visualizer2.predict([[1,68.6,46,147,92,103,75,74,1]])

visualizer2.predict([[1,88.6,66,167,112,123,95,94,1]])

from sklearn.ensemble import GradientBoostingRegressor

from sklearn.model_selection import train_test_split

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

```

```

import matplotlib.pyplot as plt

from yellowbrick.regressor import PredictionError, ResidualsPlot

# Assuming X_train, X_test, y_train, y_test are already defined

# Initialize the Gradient Boosting Regressor

model2 = GradientBoostingRegressor(random_state=42) # You can adjust parameters as
needed

# Train the model

model2.fit(X_train, y_train)

# Evaluation on training set

y_train_pred = model2.predict(X_train)

print("Mean Absolute Error (Train):", mean_absolute_error(y_train, y_train_pred))

print("Mean Squared Error (Train):", mean_squared_error(y_train, y_train_pred))


# Evaluation on testing set

y_test_pred = model2.predict(X_test)

print("R-squared (Test):", r2_score(y_test, y_test_pred))

# Plotting observed vs predicted

plt.rcParams['figure.figsize'] = (16, 6)

x_ax = range(len(X_test))

plt.plot(x_ax, y_test, label='Observed', color='k', linestyle='-')

plt.plot(x_ax, y_test_pred, label='Predicted', color='k', linestyle='--')

plt.ylabel('Remifentanil Flow')

```

```

plt.xlabel('Testing sample data')

plt.legend(bbox_to_anchor=(0.5, -0.2), loc='lower center', ncol=2, frameon=False)

plt.show()

# Yellowbrick Prediction Error Visualizer

visualizer2 = PredictionError(model2)

visualizer2.fit(X_train, y_train)

visualizer2.score(X_test, y_test)

visualizer2.poof()

# Yellowbrick Residuals Plot

visualizer = ResidualsPlot(model2)

visualizer.fit(X_train, y_train)

visualizer.score(X_test, y_test)

visualizer.poof()

# Making a prediction using the trained model

prediction = model2.predict([[0, 70.0, 52, 165, 110, 113, 95, 107, 3]])

print("Predicted Remifentanyl Flow:", prediction)

```